

Questions

1. Please modify above algorithm to reconstructing correspond shortest path.

使用一個大小亦為 $|V| \times |V|$ 的二維陣列 M 儲存中繼點，並將裡面的所有元素初始化為 -1 (亦即無中繼點)，當中的元素如 m_{ij} 代表從 i 到 j 經過的點。

```
for k from 1 to |V|
  for i from 1 to |V|
    for j from 1 to |V|
      if dist[i][j] > dist[i][k] + dist[k][j]
        dist[i][j] ← dist[i][k] + dist[k][j]
        M[i][j]=k
      end if
```

若要找出兩點中的最短路徑，可從陣列 M 找到相對應的中繼點。若找到中繼點則再尋找起點至此中繼點、中繼點至終點中是否有其他中繼點存在，若無中繼點則此兩點的直接連線即為最短距離。

```
function find_path(s, t):
  if medium[s][t] == -1:
    return
  find_path(s, medium[s][t])
  out medium[s][t]
  find_path(medium[s][t], t)
```

2. Please modify above algorithm to detecting negative cycles of G .

經過一次Floyd–Warshall即可獲得最短路徑，檢測對角線上的元素 R_{ii} 若有負權重就代表此向量圖存在負環。

3. Please answer the differences between $R_{ii} = 0$ and $R_{ii} = \text{inf}$ in Question 1.

$R_{ii} = 0$ 是初始化時表示自己與自己的距離為 0

$R_{ii} = \text{inf}$ 是初始化時預設自己與目標的距離為 inf ，亦即自己與自己之間沒有路徑，這會使此演算法無法順利運行。

4. Analyze space complexity, time complexity of best/worst case in Question 1 and Question 2
 1. find_path的複雜度為 $O(V)$
 2. 判斷負環的複雜度為 $O(V)$
5. Please solve <https://oj.csie.ndhu.edu.tw/problem/ALG06>

C++

```
#include <iostream>
#include <vector>
```

```
using namespace std;

struct Edge
{
    int a,b,d;
};

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n,m,N;
    cin>>N;
    while(N--)
    {
        cin>>n>>m;
        vector<Edge> edges(m);
        vector<int> dis(n,1001);
        for(int i=0;i<m;i++)
        {
            cin>>edges[i].a>>edges[i].b>>edges[i].d;
        }

        for(int i=0;i<n-1;i++)
        {
            for(Edge ele:edges)
            {
                dis[ele.b]=min(dis[ele.a]+ele.d,dis[ele.b]);
            }
        }

        bool nag_cycle=0;
        for(Edge ele:edges)
        {
            if(dis[ele.a]+ele.d<dis[ele.b])
            {
                nag_cycle=1;
                break;
            }
        }

        if(nag_cycle)
            cout<<"possible"<<endl;
        else
            cout<<"not possible"<<endl;

        edges.clear();
        dis.clear();
    }
}
```